

# InvoiceVault — Application Architecture

today

## InvoiceVault — Application Architecture

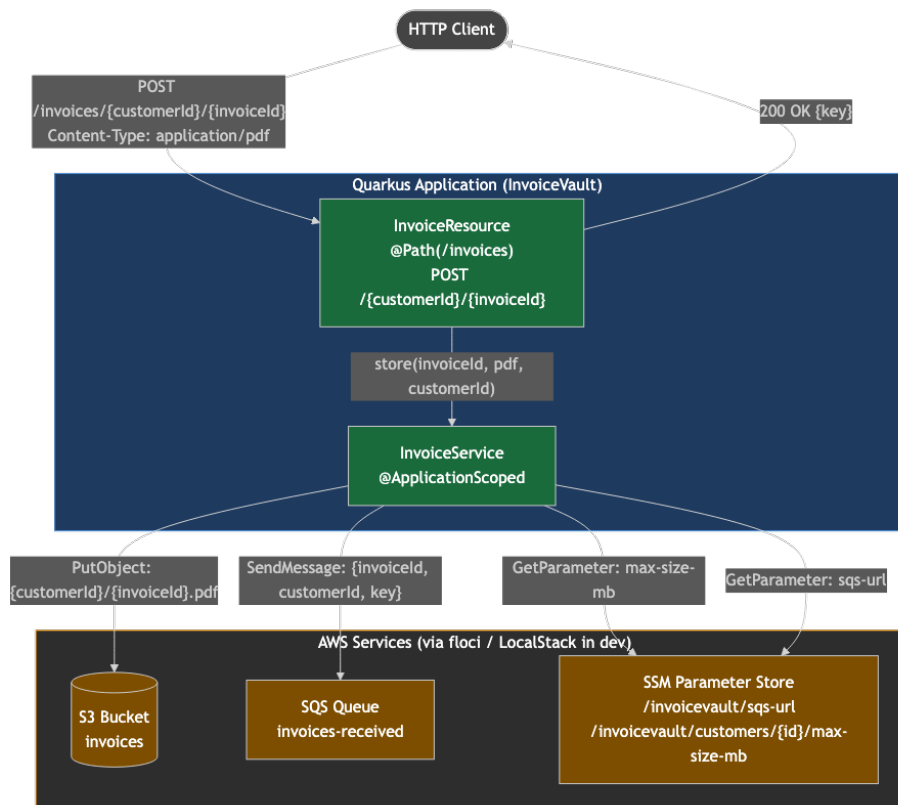
### Overview

**InvoiceVault** is a Quarkus-based REST microservice that receives PDF invoices over HTTP, enforces per-customer size limits, durably stores them in **Amazon S3**, and notifies downstream systems via **Amazon SQS**. Configuration (queue URL and per-customer limits) is managed through **AWS Systems Manager Parameter Store (SSM)**.

In the development profile the AWS services are emulated locally by [floci](#) / LocalStack using the `quarkus-aws-devservices` extension.

---

# Architecture Diagram



InvoiceVault architecture

## Component Descriptions

| Component           | Technology                                  | Responsibility                                                                                                                                             |
|---------------------|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HTTP Client         | Any HTTP client                             | Sends POST <code>/invoices/{customerId}/{invoiceId}</code> with a <code>application/pdf</code> body                                                        |
| InvoiceResource     | Quarkus REST (quarkus-rest-jackson)         | JAX-RS endpoint — validates routing, delegates to <b>InvoiceService</b> , returns the S3 object key                                                        |
| InvoiceService      | CDI<br><code>@ApplicationScoped</code> bean | Orchestrates size validation, S3 upload, and SQS notification                                                                                              |
| SSM Parameter Store | quarkus-amazon-ssm                          | Stores the SQS queue URL ( <code>/invoicevault/sqs-url</code> ) and per-customer max upload size ( <code>/invoicevault/customers/{id}/max-size-mb</code> ) |

| Component                          | Technology         | Responsibility                                                                                   |
|------------------------------------|--------------------|--------------------------------------------------------------------------------------------------|
| <b>S3 Bucket invoices</b>          | quarkus-amazon-s3  | Persistent PDF storage; objects are keyed as {customerId}/{invoiceId}.pdf                        |
| <b>SQS Queue invoices-received</b> | quarkus-amazon-sqs | Event bus; a JSON message {invoiceId, customerId, key} is published after each successful upload |

## Request Flow

1. Client sends `POST /invoices/{customerId}/{invoiceId}` with the raw PDF bytes.
2. **InvoiceResource** receives the request and calls `InvoiceService.store()`.
3. **InvoiceService** fetches the customer's `max-size-mb` limit from **SSM**.
4. If the PDF exceeds the limit, an `IllegalArgumentException` is thrown ( $\rightarrow$  `400 Bad Request`).
5. The PDF is uploaded to **S3** under the key `{customerId}/{invoiceId}.pdf`.
6. The SQS queue URL is retrieved from **SSM**.
7. A JSON notification is published to **SQS**.
8. The S3 object key is returned to the caller in a `200 OK` JSON response `{"key": "..."}.`

## Technology Stack

- **Runtime:** Quarkus 3.35.4, Java 25
- **AWS SDK:** Quarkiverse Amazon Services (S3, SQS, SSM)
- **Dev services:** floci (LocalStack-compatible emulator via `quarkus.aws.devservices`)
- **Build:** Maven with the Quarkus Maven Plugin; native image support via GraalVM